



Top 5 JavaScript Design Patterns

Every Front End Developer
Must Know



Find out more at →

greatfrontend.com

Module Pattern

The Module Pattern creates a self-contained unit of code, preventing namespace pollution and keeping code organized.

Use Case Organize related functions and variables, like managing a counter or user settings.

```
1  const Counter = (function () {
2      let count = 0; // private variable
3
4      return {
5          increment: function () {
6              count++;
7              console.log(count);
8          },
9          decrement: function () {
10             count--;
11             console.log(count);
12         }
13     };
14 })();
15
16 Counter.increment(); // 1
17 Counter.decrement(); // 0
```



Find out more at →

greatfrontend.com

Singleton Pattern

The Singleton Pattern ensures a single instance of a class, providing a global access point—ideal for shared resources like loggers or configs.

Use Case Managing global state or shared resources like a database connection.

```
1 class Logger {
2     constructor() {
3         if (Logger.instance) return Logger.instance;
4         this.logs = [];
5         Logger.instance = this;
6     }
7
8     log(message) {
9         this.logs.push(message);
10        console.log(message);
11    }
12 }
13
14 const logger1 = new Logger();
15 const logger2 = new Logger();
16
17 logger1.log('Singleton Pattern in Action');
18 console.log(logger1 === logger2); // true
```



Find out more at →

greatfrontend.com

Factory Pattern

The Factory Pattern centralizes object creation by defining an interface while allowing subclasses to determine the object type.

Use Case Creating different types of objects, such as various UI components or different shapes.

```
1 function Car(make, model) {
2   this.make = make;
3   this.model = model;
4 }
5
6 function Bike(make, model) {
7   this.make = make;
8   this.model = model;
9 }
10
11
12 function VehicleFactory() {
13   this.createVehicle = function (type, make, model) {
14     switch (type) {
15       case 'car':
16         return new Car(make, model);
17       case 'bike':
18         return new Bike(make, model);
19       default:
20         return null;
21     }
22   };
23 }
24
25
26 const factory = new VehicleFactory();
27 const car = factory.createVehicle('car', 'Toyota', 'Corolla');
28 const bike = factory.createVehicle('bike', 'Yamaha', 'FZ');
29
30 console.log(car); // Car { make: 'Toyota', model: 'Corolla' }
31 console.log(bike); // Bike { make: 'Yamaha', model: 'FZ' }
```



Find out more at →

greatfrontend.com

Observer Pattern

The Observer Pattern allows a subject to notify multiple observers about state changes. It's ideal for handling real-time events like UI updates or server responses.

Use Case Implementing a chat application where multiple users are notified of new messages.

```
1 function Subject() {
2   this.observers = [];
3 }
4
5 Subject.prototype.addObserver = function(observer) {
6   this.observers.push(observer);
7 };
8
9 Subject.prototype.notify = function(message) {
10  this.observers.forEach(observer => observer.update(message));
11 };
12
13 function Observer(name) {
14   this.name = name;
15 }
16
17
18 Observer.prototype.update = function(message) {
19   console.log(`${this.name} received message: ${message}`);
20 };
21
22 const subject = new Subject();
23 const observer1 = new Observer('Alice');
24 const observer2 = new Observer('Bob');
25
26 subject.addObserver(observer1);
27 subject.addObserver(observer2);
28
29
30 subject.notify('Hello, world!');
// Output: Alice received message: Hello, world!
//         Bob received message: Hello, world!
```



Find out more at →

greatfrontend.com

Prototype Pattern

The Prototype Pattern creates new objects based on an existing object (the prototype). It's useful for creating multiple objects with similar structure and behavior, saving resources and boosting performance.

Use Case Creating new objects that are clones of an existing one, like new user profiles with default settings.

```
1  const carPrototype = {
2    make: 'Toyota',
3    model: 'Corolla',
4    drive() {
5      console.log(`Driving a ${this.make} ${this.model}`);
6    }
7  };
8
9  function createCar() {
10     return Object.create(carPrototype);
11  }
12
13  const car1 = createCar();
14  car1.drive(); // Driving a Toyota Corolla
15
16  const car2 = createCar();
17  car2.make = 'Honda';
18  car2.model = 'Civic';
19  car2.drive(); // Driving a Honda Civic
```



Find out more at →

greatfrontend.com